

Take-Home Assignment: Multi-Rate Pricing Calculator

Estimated time: 6–8 hours

Role: Full Stack Developer (3–6 years experience)

Overview

Build a small web application where users create documents with line items, apply per-line discounts and tax rules, compute totals correctly, and view a summary report for a date range.

This exercise tests careful numeric logic, document lifecycle management, and API validation — patterns common in quotes, proposals, and billing tools. No tax compliance knowledge is required.

Requirements

1. Authentication

- Implement sign up and log in (email + password is sufficient).
- Each user must only see and modify their own data.

2. Documents

Users create **documents** with:

Field	Description
Title	Document title
Customer	Customer name
Issue date	Date the document was issued
Status	draft or finalized
Line items	See below

Each **line item** has:

Field	Description
Description	Item or service name
Quantity	Number (≥ 1)
Unit price	Price per unit (≥ 0)
Discount	Optional — fixed amount or percent
Tax	Optional — percent applied to the line

3. Calculations

Compute totals **per line**, then sum for the document.

Per line:

1. Line subtotal = quantity $\times$ unit price
2. Apply discount (fixed amount subtracted, **or** percent off subtotal — not both)
3. Apply tax percent on the **discounted** line amount

4. Line total = discounted amount + tax

Document totals:

Field	Description
Subtotal	Sum of (qty × unit price) before discounts
Total discount	Sum of all line discount amounts
Total tax	Sum of all line tax amounts
Grand total	Sum of all line totals

Important:

- All totals must be computed **server-side**. The client must not be the source of truth.
- Pick a **rounding policy** (e.g. round to 2 decimal places per line) and apply it consistently. Document your policy in the README.

4. Document lifecycle

Status	Behavior
draft	Fully editable — add, edit, remove lines
finalized	Read-only — no edits to lines, amounts, or metadata

- Provide a way to **finalize** a draft document.
- Attempts to edit a finalized document must be rejected by the API with a clear error.
- Document whether users can duplicate a finalized document into a new draft (stretch goal).

5. Summary report

Build a report filtered by issue date range showing:

- Number of documents
- Sum of grand totals
- Sum of total tax
- Sum of total discount

6. API

Expose a REST API.

- CRUD for documents and line items (respecting draft/finalized rules).
- Endpoint to finalize a document.
- Validation with specific error messages (e.g. invalid percent, negative quantity).

Sample document

Use this to verify your calculations. Expected results below assume **round to 2 decimal places per line** (tax and totals).

Line	Qty	Unit price	Discount	Tax
Widget A	2	100.00	10%	5%
Widget B	1	50.00	—	5%
Service fee	1	200.00	\$20 fixed	—

Per-line expected results:

Line	Subtotal	Discount amount	After discount	Tax amount	Line total
Widget A	200.00	20.00	180.00	9.00	189.00
Widget B	50.00	0.00	50.00	2.50	52.50
Service fee	200.00	20.00	180.00	0.00	180.00

Document expected totals:

Field	Amount	How derived
Subtotal	450.00	200 + 50 + 200
Total discount	40.00	20 + 0 + 20
Total tax	11.50	9.00 + 2.50 + 0
Grand total	421.50	189.00 + 52.50 + 180.00 (or 450 – 40 + 11.50)

Rules to confirm against this sample:

1. Discount is applied **before** tax.
2. Tax percent is applied on the **discounted** line amount (Widget A: 5% of 180, not of 200).
3. A line may have percent discount **or** fixed discount, not both.
4. Fixed discount must not exceed that line's subtotal (reject or clamp — document your choice).
5. If you use a different rounding policy, document it and keep results internally consistent.

Stretch goals (optional)

- **Duplicate** — copy a finalized document into a new draft.
- **Finalize validation** — reject finalize if any line has quantity ≤ 0 or negative prices.
- **Printable view** — HTML or PDF output of a document.

Technical guidelines

- **Stack:** Your choice.
- **Deployment:** Required — include a live URL to a deployed version of the app.
- **Tests:** Unit tests on the calculation module are **strongly appreciated** — this is the highest-value test surface in this assignment.
- **Money handling:** Avoid floating-point drift. Use a strategy appropriate to your language (e.g. integer cents, decimal library).

Deliverables

Submit a Git repository containing:

1. **Source code** — backend, frontend, and any migrations/seed scripts.
2. **Deployed live URL** — a publicly accessible link to the running app.
3. **README** with:
 - Prerequisites and step-by-step setup.
 - Calculation and rounding policy (with a worked example).
 - Finalize/immutability rules.
 - Assumptions and tradeoffs.
 - What you would improve before production.

- The deployed URL (also include it in your submission email).

Optional: a short Loom/video walkthrough (5–10 minutes).

What we evaluate

Area	What we look for
Correctness	Line and document totals correct for mixed discount/tax
Calculation design	Single shared module; consistent rounding
Lifecycle	Finalized docs immutable via API
Validation	Specific errors for bad input
Reporting	Summary totals match individual documents in range
Tests	Calculation unit tests
Communication	README clarity, especially rounding policy

Questions?

If anything is ambiguous, make a reasonable assumption, document it in your README, and proceed.

Good luck — we look forward to reviewing your work.